BUILDERS HUB · 23 OCT 2026

Cross-platform apps: iOS & Android with Claude and Modaal Studio

Why the architecture decides how far an app can grow.
+ LIVE BUILD: two apps with twin Swift<>Kotlin cores that share recordings.

Ivan Misuno · co-founder, Modaal.dev

• SPEAKER

I’m Ivan Misuno, co-founder of Modaal.dev (ex-Uber, Facebook)

Before Modaal we ran a mobile agency and an indie app studio, PopAppFactory (2009–2014):

50+
educational kids’ apps

6M
downloads

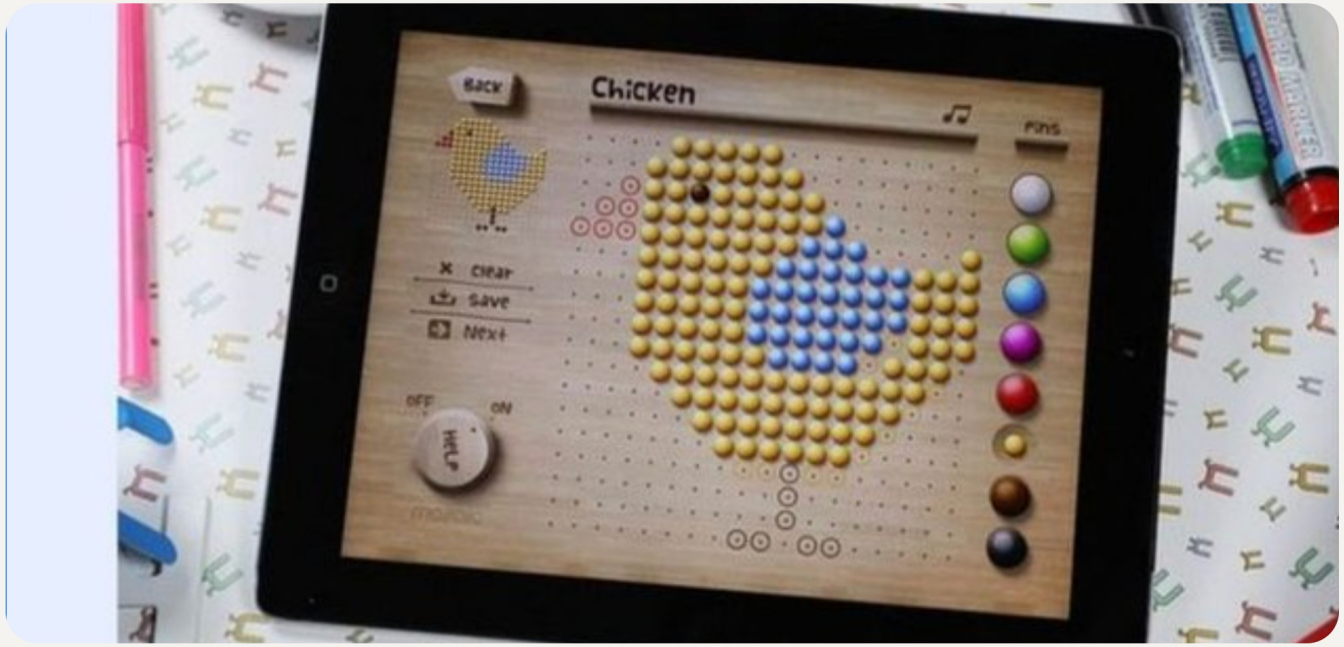
$1M
in revenue

Apple App of the Year
Innovation category



POPAPPFACTORY · 2009–2014

Our apps and games



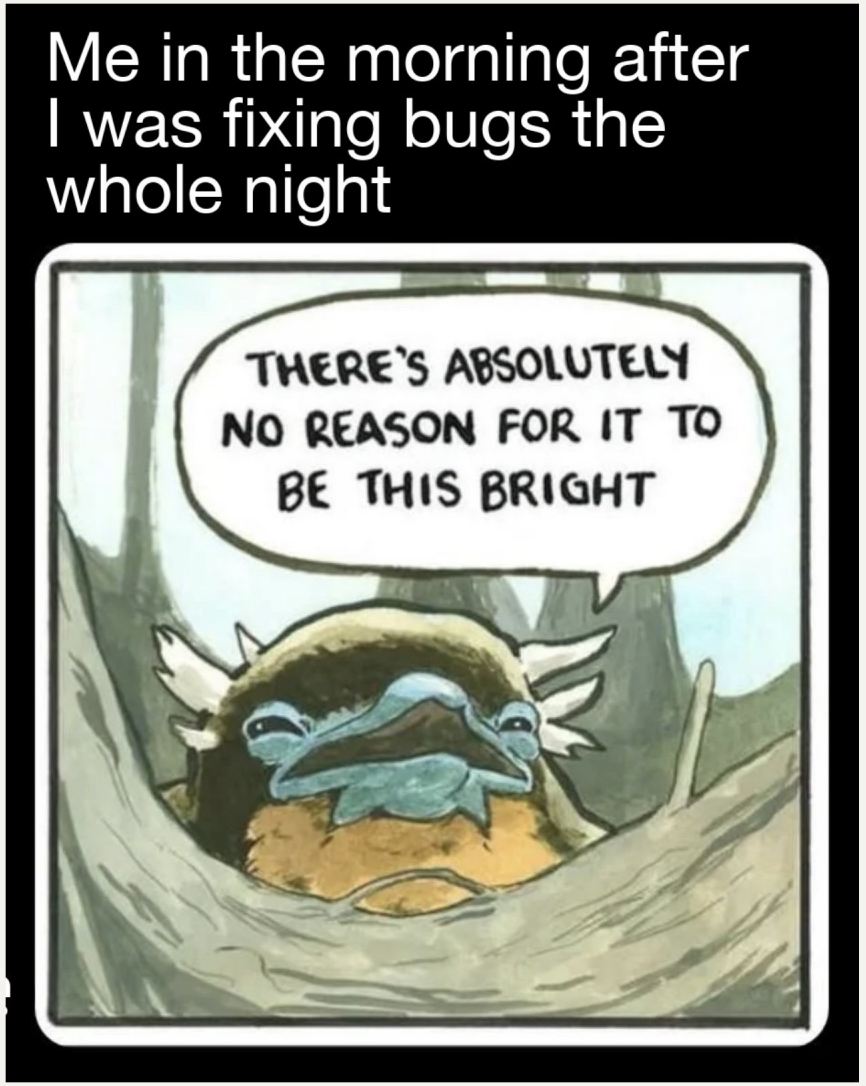
Mosaic for iPad · 2010



50+ App Store titles

Managing a portfolio of apps is hard


Tomorrow is deadline
Project*
Developer*
Tomorrow Morning!


Me in the morning after
I was fixing bugs the
whole night
THERE'S ABSOLUTELY
NO REASON FOR IT TO
BE THIS BRIGHT

What MVC became in our apps

A shared, mutable data model

Singletons and a global game state.

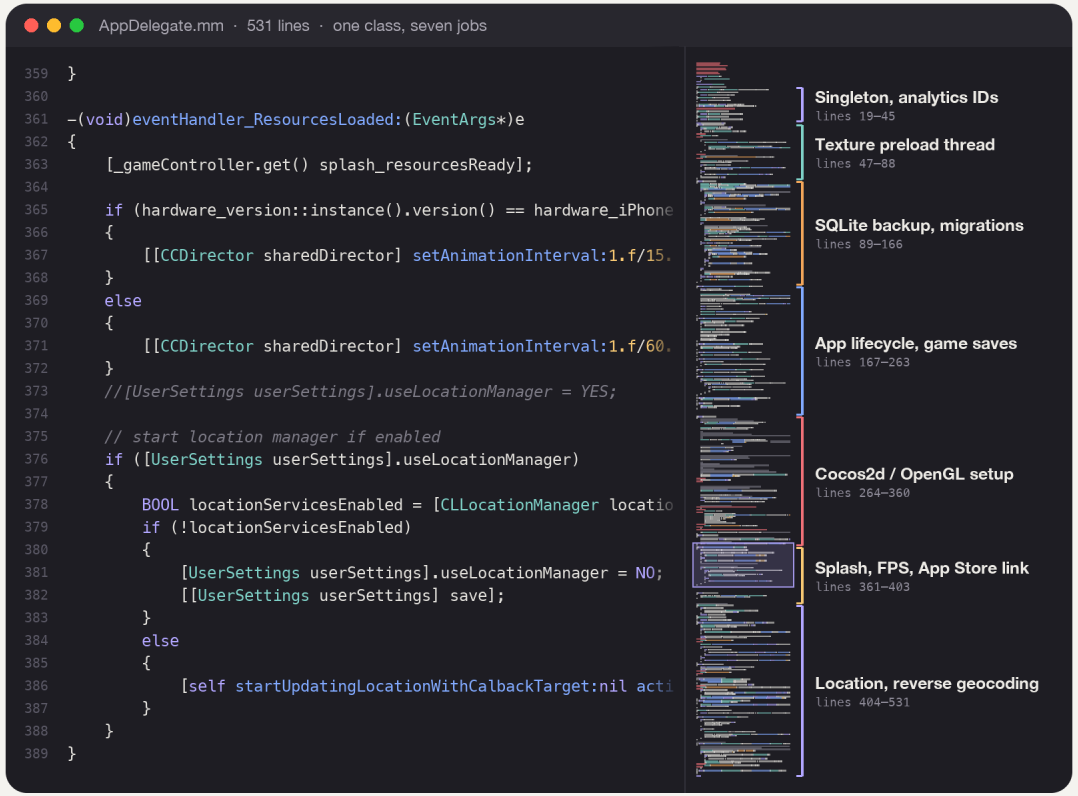
Data and control in both directions

Delegates, NSNotificationCenter, KVO, and view controllers calling each other.

No compile error, no failing test

A change in one screen could alter behavior in another.

Telltale: large convoluted AppDelegate (500+ LoC)



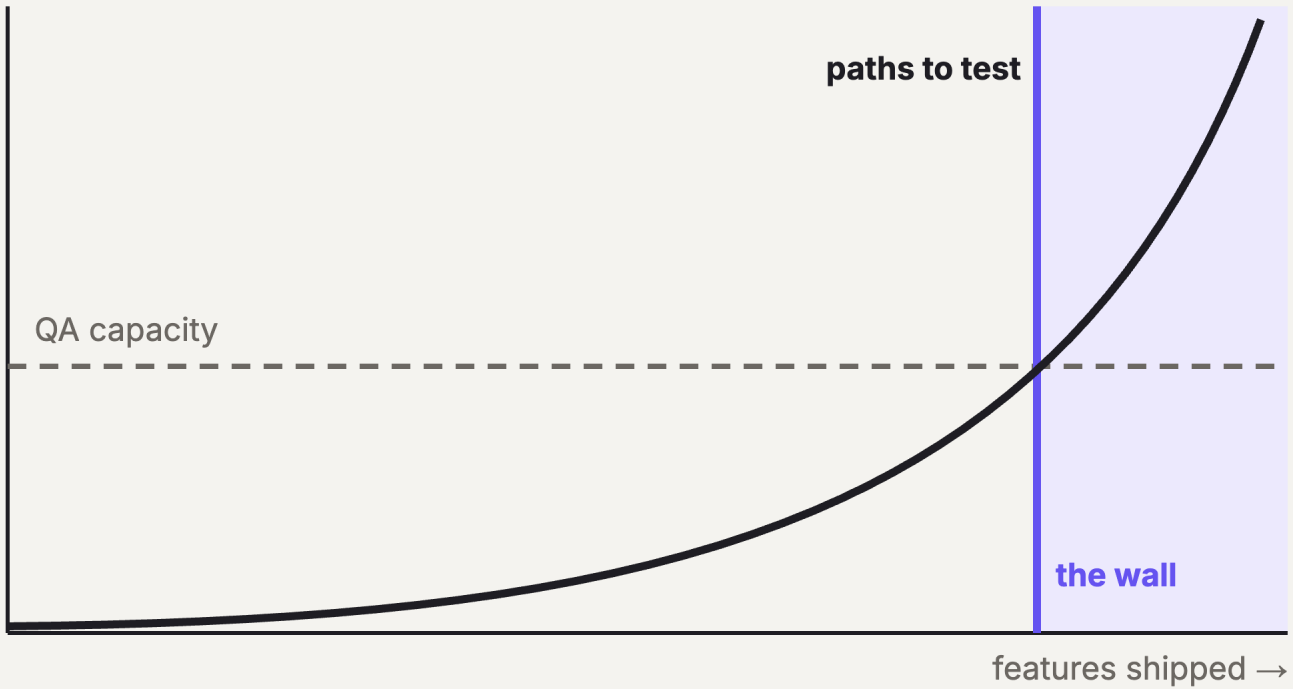
POPAPPFACTORY

The complexity wall

n state variables combine in up to 2^n ways.

QA covers a fixed number of paths per release.

The result: bug-fixing nights before every submission.



The architecture decides whether your project slows down after weeks of iteration.

Architecture is not the framework

Architecture:

Where new class goes · Where state lives · Data & Control flow · Feature isolation

Framework (UIKit · Cocos2d-objc):

Views, events and lifecycle callbacks · a scene graph and a game loop

MVC (Model-View-Controller) is not an architecture.
In practice: Massive View Controller

15:33

Aleksa Simic likes this

Oleg Kasarin · 2nd
iOS Product Engineer | Swift, SwiftUI | P...
4h · + Follow

Modern iOS app architecture, 2026 😄

We used to argue about what's better: MVVM, MVP, VIPER, TCA or Clean Architecture. Layers and protocols for every little thing.

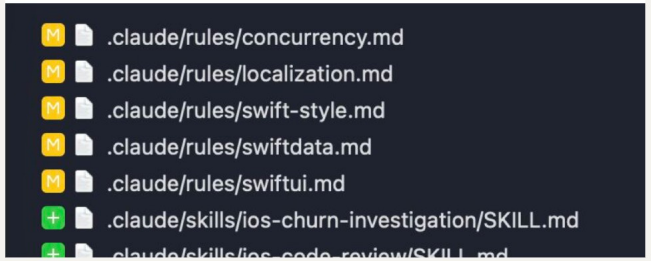
Now the "architecture" of my product looks like this 👆

A bit of context: Claude has rules and skills.
Rules are project conventions the agent picks up automatically. Mine cover Swift style, SwiftUI, Concurrency, SwiftData and localization.
Skills are instructions for specific tasks, and the agent pulls them in on its own:
- ios-code-review - code review
- ios-churn-investigation - digging into why subscribers churn
- swiftui-liquid-glass - the new Liquid Glass design from iOS 26
- swiftui-iphone-duo - adapting the UI for the new foldable iPhone Duo

I wrote it all down once and now I don't have to repeat it in every prompt. I use it every day, and it's a massive productivity boost.

Architecture isn't dead, of course. The agent just remembers it for me now 🤓

Want the same setup for your iOS project? DM me - I'll help you set up rules and skills
#iOSDev #SwiftUI #ClaudeCode



Architecture:
Moves rules out of reviewers' heads and into the type system, lint, or a test.

ARCHITECTURES

iOS

Pattern	What it moves out of the view	What still depends on review	How boundaries are enforced
MVVM	Presentation logic, into a testable view model	Navigation, cross-screen state, and what a service port does	By convention
Clean	Layers, plus the dependency rule	The layer boundaries	By convention (code review)
VIPER	Per-screen modules with protocols between their parts	Boilerplate per screen, and the wiring between modules	By convention (protocols, wired by hand)
RIBs	Business logic into a tree of Interactors; routing into Routers	Boilerplate per RIB; iOS and Android kept in step by hand	By convention (Swift protocols, explicit DI)
TCA	State, actions and a pure reducer: one direction, testable	Effects are closures that can’t be compared; Swift only	By the compiler inside the store tree; dependencies are implicit (@Dependency)

Android

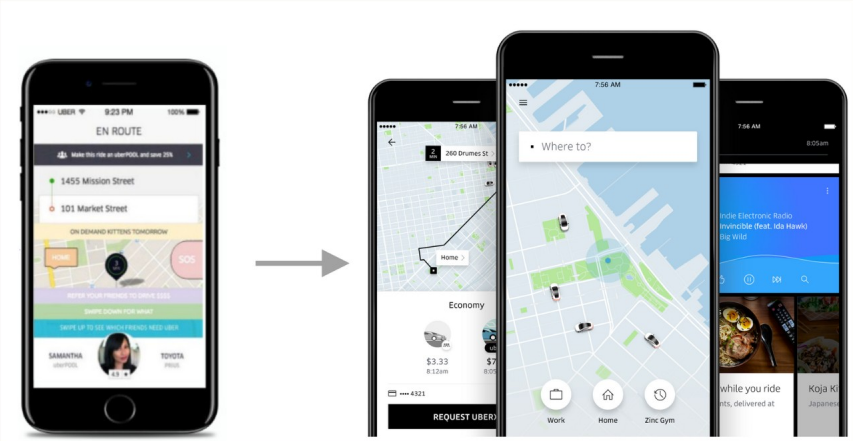
Android architecture	iOS counterpart	How parity is enforced
Activity-centric MVC (“god Activity”)	MVC (“massive view controller”)	None
MVP (Mosby)	MVP, VIPER	By convention
MVVM + Jetpack ViewModel	MVVM (ObservableObject)	By convention
Clean + Hilt / Dagger	Clean (+ hand-written DI)	By convention
MVI (Orbit, Mosby MVI)	TCA, ReSwift	By convention
RIBs (uber/RIBs)	RIBs (uber/RIBs-iOS)	By convention: two implementations
Workflow (Square) · Mobius (Spotify)	workflow-swift · Mobius.swift	By convention: same API, two codebases
KMP + Decompose / MVIKotlin / Circuit	Same Kotlin, compiled for iOS	By construction: one implementation

uber/RIBs, RIBs-iOS

One app, hundreds of engineers

RIBs came out of Uber's 2016 rider-app rewrite.

The question was: how do you allow 40 teams to cooperate in a single app?



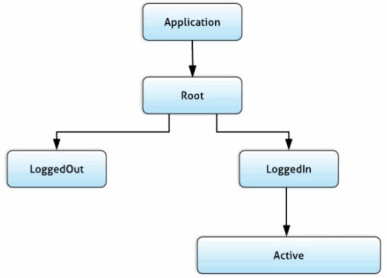


RIBS

The RIB tree

A RIB tree

Uber Engineering, 2018



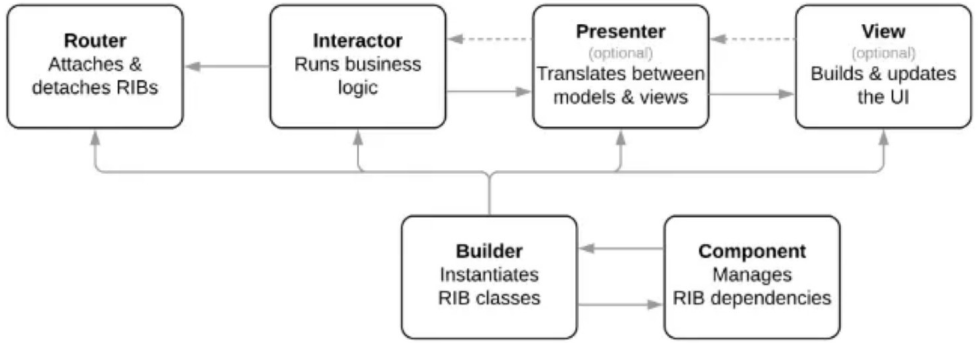
The tree follows business state, not the screens.

Sign in: **LoggedOut** detaches, **LoggedIn** attaches.

A screen is a leaf, not an owner.

RIB parts

TechAhead, 2024



Interactor decides; **Router** attaches and detaches children.

Builder creates the RIB; **Component** owns its dependencies.

Presenter and **View** are optional: a RIB can have no screen.

RIBS

Five principles

1

Boundaries are Swift protocols

Each RIB declares a Dependency protocol with exactly what it reads.

2

Static DI, checked by the compiler

A Component satisfies it; a Builder builds the RIB. “If it builds, it works.”

3

The framework owns lifecycles

Attach, detach, activation and workers are the framework’s job, not feature code’s.

4

One direction for data and control

Routing goes down, listener calls go up, nothing goes sideways.

5

Every boundary can be mocked

Swift can’t proxy a protocol at runtime, so mocks are generated (Sourcery).

These five come back in Duet.

RIBS

What RIBs didn’t solve

101 vs 28 LoC

Ceremony code

RxSwift

Obsolete since Swift.Combine (~2020)

iOS ≠ Android

Two parallel implementations, kept in step by convention. Nothing proved they behaved the same.

Architecture:
Is the question even relevant in the era of AI coding agents?

Give an agent 20 tasks, it will invent 20 different solutions

AGENTS

Agents hit the same wall, higher up

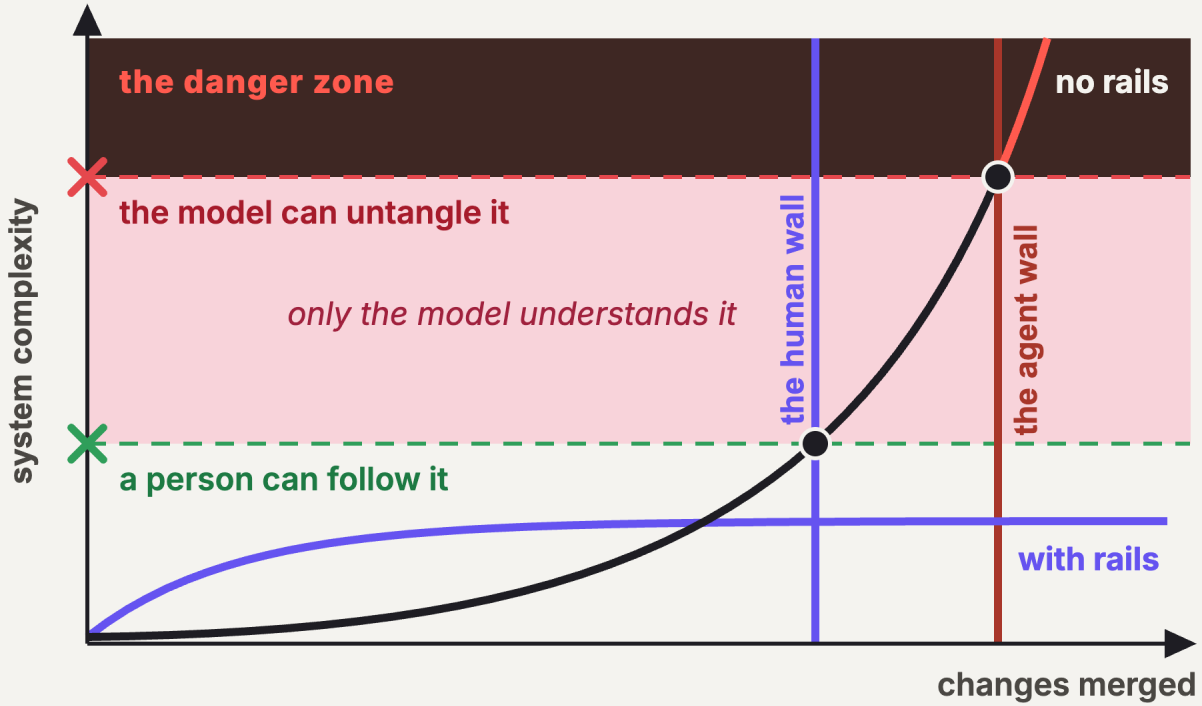
Smarter models push "the complexity wall" to the right, **past the human ability** to untangle the system.

Models **overfit tests** to the edges of the **monolith**:

- bend failing tests to stay green,
- generalization suffers.

Module decomposition + **enforced boundaries** ⇒ complexity stays constant within a single slice of the system.

Architectural guidance and guardrails matter even more with agents.



DUET

Duet principles: RIBs + Cross-verification

1
Boundaries are interfaces

2
Static DI

3
Framework-owned lifecycles

4
One-way flow

5
Generated mocks, both platforms

6
Pure reducers, effects as data

7
Navigation is state

8
Behavior recorded and replayed

```
spec → scenario → duet record → reducer → shells → duet verify → CI: record --check
```

DUET

Duet combines well-known ideas

Principle	Notable prior art	What Duet does differently
Pure reducers, effects as data	Elm (Model, Cmd Msg) · redux-loop · Elmish · re-frame · redux-saga · Spotify Mobius (Java and Swift). Counter-example: TCA's effect closures.	Effects are serialized sum types, compared byte for byte in Swift and Kotlin
Navigation is state	Elm Browser.application · SwiftUI NavigationStack(path:) · TCA StackState · Jetpack Navigation 3 · Decompose ChildStack · Flutter Router · Square Workflow	One set of navigation blocks for both platforms (path, slot, tabs, stage), gated in the recordings
Behavior recorded and replayed	One implementation: Elm debugger · Redux DevTools · event sourcing · snapshot tests. Across implementations: Pact · test262 · WebAssembly spec tests	Scenario recordings replayed by every core over one protocol; the mutation drill tests the recordings

DUET

How do two apps agree?

Option	Who draws	Logic	How the apps agree
Flutter	Framework canvas	Dart runtime	By construction (platform channels excluded)
React Native	Native views (Fabric)	JS/TS on Hermes	By construction (native modules excluded)
Compose Multiplatform	Framework canvas on iOS	Kotlin	By construction, UI included
KMP + MVI (Decompose, Circuit)	Per team	Kotlin	Shared unit tests; no gate between the apps
Skip	SwiftUI transpiled to Compose	Swift	The transpiler
Two native apps	Platform, twice	Twice	By convention: review and QA
Duet	Platform: SwiftUI + Compose	Kotlin once, or Swift + Kotlin twins	By verification: recordings replayed by every core

DUET

Three layouts, one framework

Single app

Apple: Swift core + SwiftUI shell.
Android: Kotlin/JVM core, Jetpack-Compose UI.
No alien toolchain in apps.

Shared core (KMP)

KotlinMP core written once.
Android: Compose shell links KMP core.
Apple: SwiftUI shell links KMP core via SKIE/XCFramework.

Twin cores

A Swift core for iPhone and a Kotlin/JVM core for Android, sharing only the recordings.

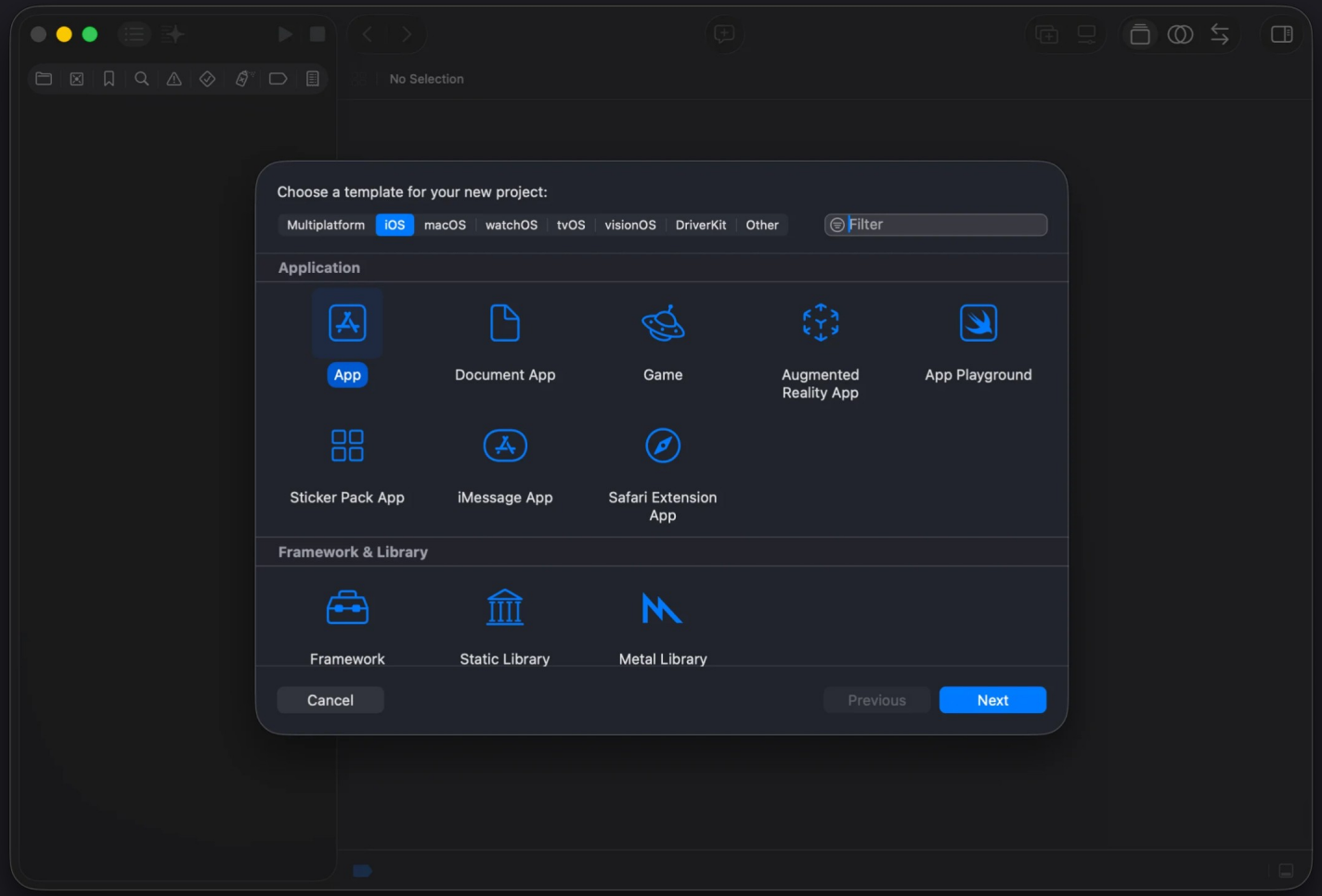
VS CODE + CLAUDE CODE

Live demo

A Swift core, then a Kotlin twin that replays the same recordings. No IDE wizard.

1 A Swift core from a spec
2 Scenario, record, replay, break
3 Add a Kotlin/JVM core
4 The two cores disagree
5 Generated mocks and a test
6 Two apps and CI

Open Xcode, New Project…



(joking) ;)

Workspace and agent rules

Scripted is the new moat.

```
$ git clone https://github.com/modaal-agent/duet-tutorials
$ cp -R duet-tutorials/tutorial1-swift-start foyer
$ code foyer
```

`parity/manifest.yaml`	features and recordings
`parity/feature-specs/`	one page per feature
`parity/fixtures/`	recordings, build products
`src-ios/Subtrees/`	one package per feature
`tools/duet`	the CLI, pinned 0.34.0

AGENTS.md

- Reducers are pure: no clock, no I/O, no platform imports.
- The Swift scenario is the only writer of recordings.
- Never edit parity/fixtures/. Re-record with tools/duet record.
- Kotlin fails a recording? Fix the Kotlin twin, never the fixture.
- Annotated protocols get generated mocks, never hand-written ones.
- Done = lint, verify, record --check, mocks --check, protocol-run --platform both.

• LIVE DEMO · 1

A Swift core from a spec

parity/feature-specs/splash.md

STATE

isArmed

ACTIONS

appeared
ceremonyFinished
safetyNetElapsed

EFFECTS, AS DATA

armSafetyNet(3000)
notifyListener(completed)

Claude Code writes the Swift package, the reducer and the manifest row. Then:

```
$ tools/duet lint
lockstep-lint: FAIL
✗ [splash] scenario file missing on disk:
…/SplashScenarioTests.swift
✗ [splash] fixture listed but missing on disk:
splash.ceremony-completes — just declared? run: tools/duet
record --feature splash
✗ … the same for three more fixtures
```

The tool names the agent's next step.

LIVE DEMO · 2

Record, replay, break

```
$ tools/duet record --feature splash
duet record: rewrote 4 fixture(s)
$ tools/duet verify
duet verify: 4/4 fixture report(s) passed · PASS
# delete in SplashFeature.swift: guard !state.isArmed else { return [] }
$ tools/duet verify
duet verify: 3/4 fixture report(s) passed · FAIL
✗ fixture 'splash.repeat-appear-inert' step 1 'repeat appear inert › nothing: the net is not re-armed' — Then
failed [swift]
at expectedEffects[0]
expected: ⌀ actual: {"id":"splash.safetyNet", … "afterMillis":3000}
```

A recording is a row of the spec, as a check that runs.

LIVE DEMO · 3

Add the Kotlin/JVM core

parity/manifest.yaml

```
replayRunner: src-ios/Libraries/Replay
kotlinCore: jvm
features:
splash:
swift: …/SplashFeature.swift
kotlin: src-android/…/SplashFeature.kt
scenario: …/SplashScenarioTests.swift
```

src-android/ is a plain Gradle build: **kotlin.jvm**, not kotlin.multiplatform.

```
$ tools/duet lint
lockstep-lint: OK
$ tools/duet verify
duet verify: 8/8 fixture report(s) passed · PASS
```

The twin is written against recordings that already exist.

LIVE DEMO · 4

The two cores disagree

```
# Swift reducer: safetyNetMillis = 2_500
$ tools/duet record && tools/duet verify
duet verify: 4/8 fixture report(s) passed · FAIL
✗ fixture 'splash.both-paths-notify-twice' step 0 'the splash appears' — effects diverged [kotlin]
at expectedEffects[0].payload.value.afterMillis
expected: 2500 actual: 3000 # and 3 more recordings
# Claude Code: fix the Kotlin twin only; don't touch parity/fixtures or src-ios
$ tools/duet verify
duet verify: 8/8 fixture report(s) passed · PASS
```

The failure names the side, the recording, the step and the field.

• LIVE DEMO · 5

Generated mocks and a test

```
/// sourcery: CreateMock
protocol SplashEnvironment { … }
```

↓ tools/duet mocks

`SplashEnvironmentMock`

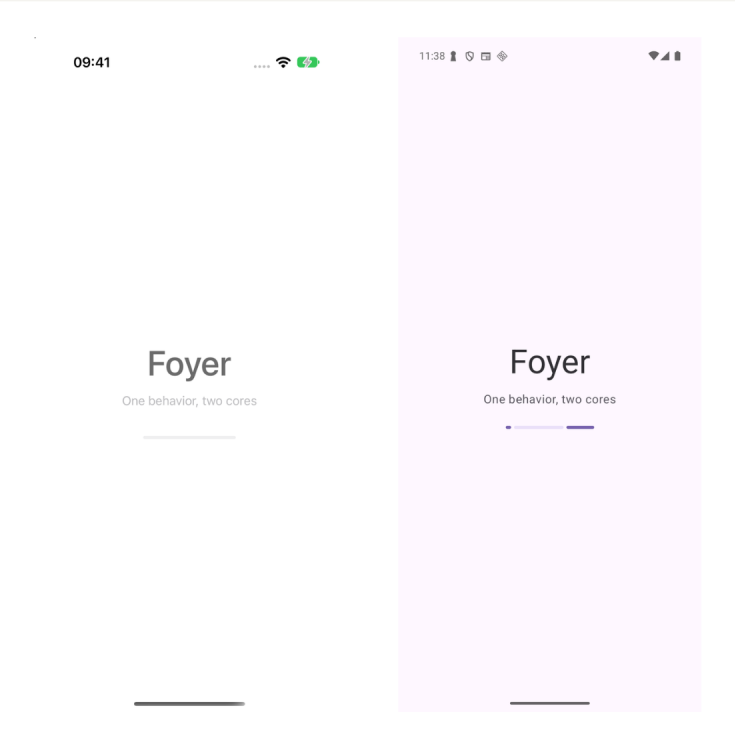
57 generated lines in
Generated/SplashFeatureMocks.swift

```
$ tools/duet mocks --check
Generated code is stale.
$ tools/duet mocks && tools/duet mocks --check
SplashFeatureMocks.swift — 1 types, 57 lines
Generated code is current ✓
$ swift test
Executed 6 tests, with 1 failure
# Claude Code finishes the stub with the generated mock
$ swift test
Executed 6 tests, with 0 failures
```

No hand-written double, because one is already generated.

LIVE DEMO · 6

Two apps, one set of recordings



The iPhone app links no Kotlin

Release build: 2,310 symbols, none with Kotlin/Native's kfun: prefix. No Frameworks directory.

CI: scripts/run-tree.sh

`lint · doctor · verify · record --check · mocks --check · protocol-run --platform both`

macOS lane: swift test + xcodebuild. Ubuntu lane: :app tests + assemble.

Shared core vs. twin cores

	Shared core (KMP)	Twin cores
Feature logic	Once, in commonMain	Twice: Swift + Kotlin/JVM
Kotlin module plugin	kotlin.multiplatform	kotlin.jvm
iPhone app links	FoyerKit.xcframework (Kotlin/Native)	Swift packages only; 0 kfun: symbols
iOS build	assemble_kit.sh (about 10 min the first time), then xcodebuild	xcodebuild only, no Gradle step
Recordings written by	The Kotlin scenario	The Swift scenario; Kotlin only verifies
duet verify lanes	Kotlin 4/4, plus a separate Apple boundary lane	Swift + Kotlin, 8/8
Edits per behavior change	One reducer	Two reducers, both checked by the recordings

SplashFeature.kt is the same file in both layouts.

MOCK CLASSES

Generated vs. hand-written mocks

	Swift, hand-written	Swift, generated	Kotlin, hand-written	Kotlin, generated
cost	$24.99	$22.11	$16.47	$16.97
budget tokens	1,617,123	1,497,001	1,009,424	1,099,439
turns	113	103	98	87
tests added	142	141	143	145
lines added	3,796	2,902	3,218	2,507
hand-written doubles	17	0	16	0
mutation score	0.797	0.750	0.641	0.594

Same cost and test output · about 23% fewer lines to review · zero hand-written doubles

claude-opus-5 high · Claude Code 2.1.271 · one 8-prompt session over an 8-class subtree of a real app · n = 3 per cell, medians · modaal-agent-mock-lab RESULTS.md, round 5

HOW MUCH DOES THE ARCHITECTURE COST IN TOKENS

The cost of keeping an agent on rails - WIP

What the rails cost

- The guidance itself: a skill ≈ 286 tokens resident, ≈ 4.8k on invoke
- Twin cores: a second edit per behavior change
- CI time: the mutation drill takes 5–9 s per mutant
- A spec takes longer to reach the first running screen

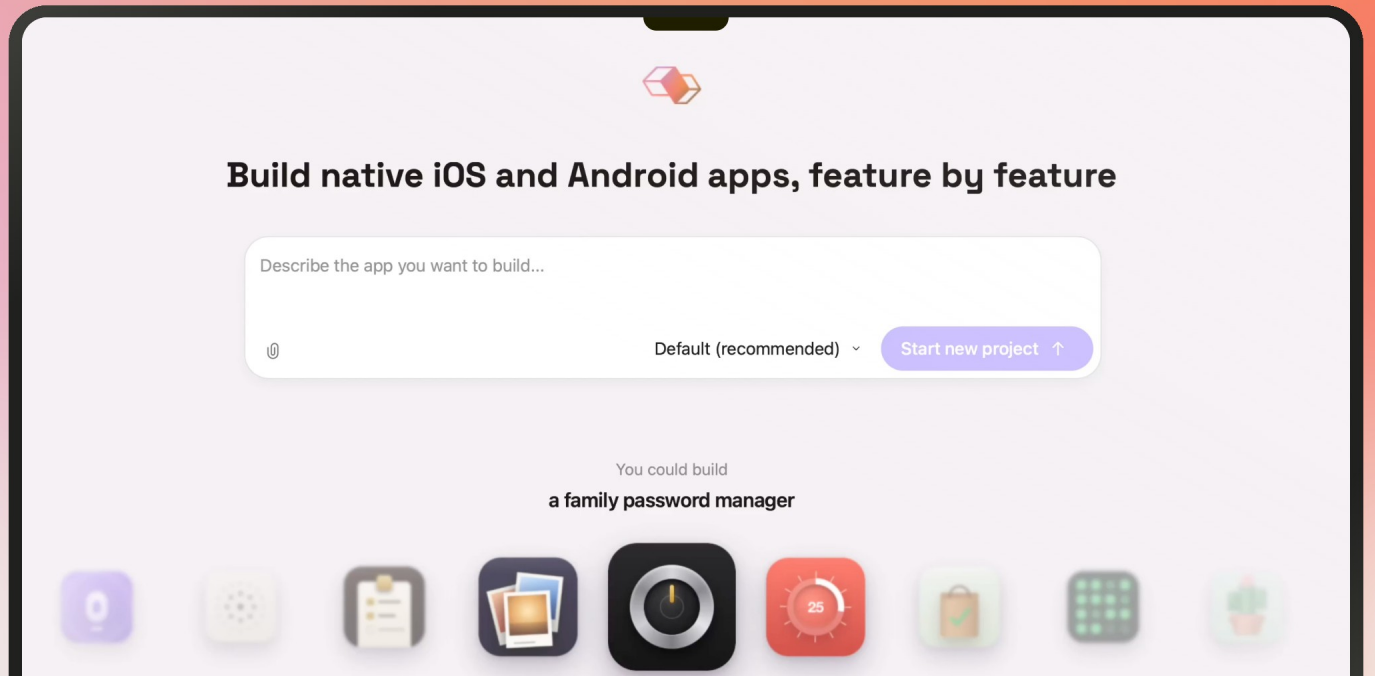
What drift costs

- Lab run with no CLAUDE.md: 0 of 64 mutants killed, $20.46 lost
- “All 117 tests pass”, while the suite printed 4 failures
- Authoring tools, same gate: −36.7% budget tokens (n = 3)
- Memory Lane → Android: 134/134 gates, zero shared-logic edits

Modaal Studio

Build and release native Swift and Kotlin apps with AI

modaal.dev



MODAAL STUDIO · FROM ONE SENTENCE

Modaal scaffolds initial project structure + sets up guardrails and CI

1

Repository and Duet tree

Modaal creates the folder, the git repository and the Duet tree.

2

Plan first

It sends your prompt with Plan on.

3

PRD and spec

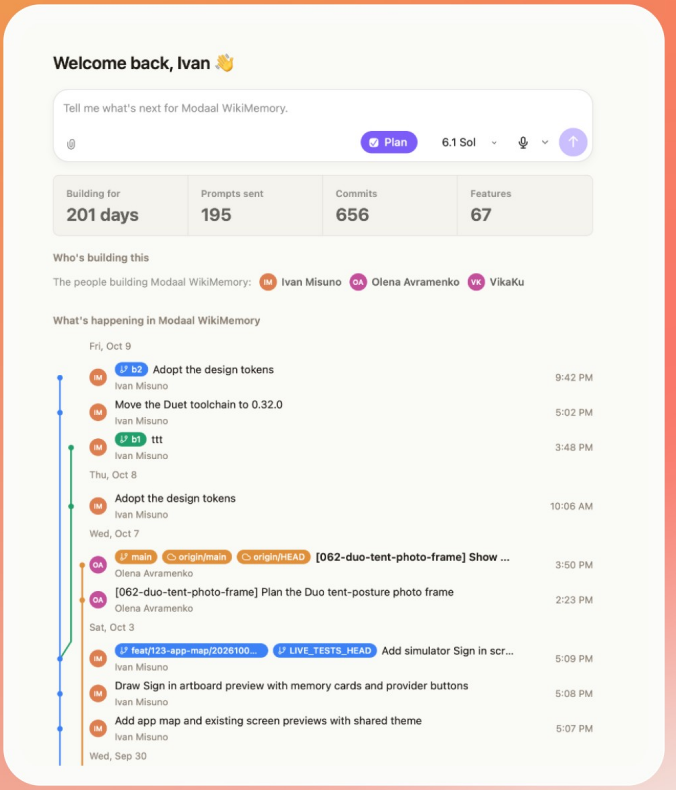
The agent writes PRD.md and specs/001-<feature>/spec.md.

4

Architecture on file

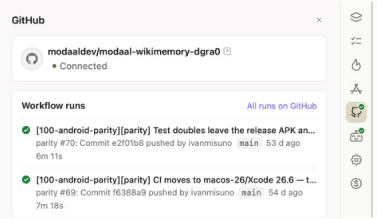
.modaal/project.json records the architecture, so the agent never asks.

```
src-ios/App/ src-ios/Subtrees/<Name>/ src-ios/Libraries/
parity/ tools/duet .github/workflows/
AGENTS.md PRD.md specs/ .modaal/project.json
```



MODAAL STUDIO · BEYOND THE CODE

What Modaal does beyond the code.



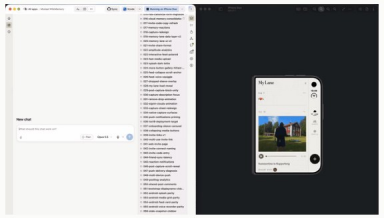
Workflows/CI

Check status on every commit; a Bitrise panel and Build Hub.



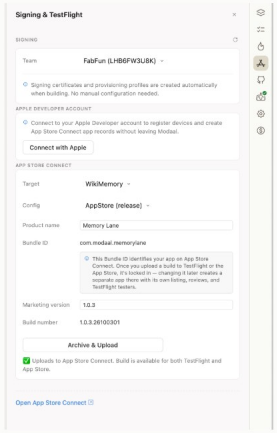
Design rules

Pixel-accurate rendering
Shared design tokens



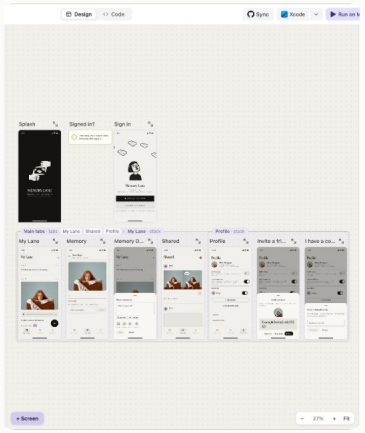
Debugging

The agent reads logs, screenshots and the UI tree; it taps, types, uses LLDB.



TestFlight

Archive and upload, App Store record creation, “Fix with agent”.



Design and app map

Every screen on an artboard, grouped by navigation.

Questions

The architecture decides whether change #200 costs what change #20 did.

github.com/modaal-agent/duet
github.com/modaal-agent/duet-tutorials
docs.modaal.dev/tutorials/duet
modaal.dev